

“스케이트 연습” 문제 풀이

작성자: 오주원

서술의 편의를 위해 시작 지점, 도착 지점 대신 0번 지점, $N+1$ 번 지점이라고 하고, $V_0 = V_{N+1} = 0$ 이라고 하자. 또한, 각 부분문제에서의 V 의 상한을 K 라고 하자.

부분문제 1

1만큼만 낮추는 경우를 고려하지 않고 N 개의 지점에서 가질 수 있는 속력은 $O(K^N)$ 가지가 있을 수 있다. 찾은 $O(K^N)$ 가지 방법을 $O(N)$ 으로 가능한지 판별해주고, 가능한 방법 중 속력의 합이 최대인 경우를 출력한다. 따라서, 전체 문제를 $O(K^N \times N)$ 에 해결할 수 있다.

부분문제 2

다음과 같이 $DP\ D[i][j]$ ($0 \leq i \leq N+1, 0 \leq j \leq K$)를 정의하자.

- $D[i][j] := i$ 번 지점에서 j 의 속력을 가졌을 때 가능한 1번 지점부터 i 번 지점까지의 최대 속력 합

i 번째 지점에서 j 의 속력을 만들기 위해선 이전 지점에서 0 이상 $V_i + 1$ 이하의 속력을 가지고 있어야 하므로 다음과 같이 점화식을 세울 수 있다.

- $D[0][0] = 0$
- $D[i][j] = \max_{0 \leq k \leq j+1} \{D[i-1][k] + j\}$ ($1 \leq i \leq N+1, 0 \leq j \leq V_i$)

$N+1$ 번째 지점에서 0의 속력으로 끝나야 하므로 답은 $D[N+1][0]$ 에 들어 있다. 중간 지점에서 속력이 0이 되는 것보다 1이 되는 것이 항상 이득이므로 0이 되는 경우는 따로 처리해주지 않아도 된다. 따라서, 문제를 $O(NK^2)$ 에 해결할 수 있다.

부분문제 3

부분문제 2의 풀이를 가져오자. 여기에 추가로 $DP\ P[i][j]$ ($0 \leq i \leq N+1, 0 \leq j \leq K+1$)를 정의하자.

- $P[i][j] := i$ 번째 지점에서 0이상 j 이하의 속력을 가졌을 때 가능한 1번 지점부터 i 번 지점까지의 최대 속력 합

부분문제 2에서 $D[i][*]$ 의 값을 구한 후, $D[i+1][*]$ 의 값을 구하기 전에 다음과 같이 $P[i][*]$ 를 구한다.

- $P[i][0] = D[i][0]$
- $P[i][j] = \max(P[i][j-1], D[i][j])$ ($1 \leq j \leq V_{i+1} + 1$)

$P[i-1][*]$ 을 구했다면, 다음과 같이 $D[i][*]$ 의 점화식을 세울 수 있다.

- $D[i][j] = P[i-1][j+1] + j$ ($1 \leq j \leq V_i$)

부분문제 2와 D 의 정의는 바뀌지 않았으므로 $D[N + 1][0]$ 에 문제의 답이 들어있다. $D[*][*]$ 와 $P[*][*]$ 의 모든 값은 각각 $O(NK)$ 에 구할 수 있으므로, 문제를 $O(NK)$ 에 해결할 수 있다.

부분문제 4

i 번째 지점에서 $N - i + 1$ 이상으로 속력을 가진다면 도착 지점에서 0의 속력을 가질 수 없다. 그러므로 각 부분문제 3 풀이의 V_i 를 사용하는 부분에 $\min(V_i, N - i + 1)$ 을 대신 사용해주면 문제를 해결할 수 있다. V 의 상한이 $K = 10^9$ 가 아닌 N 이 되므로 시간복잡도는 $O(N^2)$ 이다.

부분문제 5

i 번째 지점에서 가질 수 있는 최대 속력을 M_i 라고 하자. 이 값은 $M_{i+1} + 1$ 과 V_i 중 더 작은 값이다. 도착 지점을 속력 제한이 0인 $N + 1$ 번째 지점이라고 생각하고 M_{N+1} 을 0으로 둔 뒤, N 부터 1까지 차례대로 M_i 을 채워주면, 모든 중간 지점에서의 최대 속력이 구해지므로 M 에 들어있는 값들의 합이 문제의 정답이 된다. 시간복잡도는 $O(N)$ 이다.

“지그재그” 문제 풀이

작성자: 김준원

부분문제 1

가장 단순한 방법으로 문제를 푼다. $f(x, y, z)$ 를 구하기 위해, 수열 A 의 가능한 모든 부분수열을 직접 살펴보면서 조건 1., 2., 3.을 만족하는지 확인한다. 길이 N 인 수열 A 의 부분수열은 총 2^N 가지 존재한다(각 원소에 대해서 2가지 선택지 - ‘고른다’와 ‘고르지 않는다’가 도착하므로). 각각에 대해 반복문을 한 번 이용해 $O(N)$ 으로 세 조건을 모두 판별할 수 있다. 세 조건을 모두 만족하는 부분수열 중 가장 긴 부분수열의 길이가 $f(x, y, z)$ 가 될 것이다. 함수 f 의 $O(N^3)$ 가지 값을 모두 구해야 하므로, 총 시간 복잡도는 $O(N^4 2^N)$ 이다.

어떤 수열이 조건 1., 2.를 만족하는지는 쉽게 판별할 수 있으므로, 두 조건을 만족하는 부분수열만을 순회해서 조건 3.을 검사하는 방법도 있을 것이다.

부분문제 2

부분문제 1에서는 총 2^N 가지의 부분수열을 일일이 순회했다. 여기서 병목은 조건 3.이었는데, ‘어떤 수열의 가장 긴 지그재그 부분수열’을 더 빠르게 구할 수 있는 방법이 존재한다면 시간 복잡도를 크게 개선할 수 있을 것이다.

길이 K 인 수열 $S_1, \dots, S_K$ 의 가장 긴 지그재그 부분수열은 다음과 같은 간단한 그리디 알고리즘을 이용해 구할 수 있다.

1. 가장 긴 지그재그 부분수열을 담을 배열 arr 를 준비한다. 최초에 arr 은 빈 배열이다.
2. $i = 1, 2, \dots, K$ 를 순서대로 순회하며,
 - (a) arr 의 끝에 S_i 를 추가한 뒤에도 arr 이 지그재그 수열이라면, arr 의 끝에 S_i 를 추가한다.
 - (b) 그렇지 않다면, arr 의 마지막 원소를 S_i 로 대체한다.

이 알고리즘의 정당성은 직관적으로도 눈에 보이지만, 아래 부분문제 4에 대한 문단에서 구체적으로 논의할 것이다.

이로써 각 x, y, z 에 대해 $f(x, y, z)$ 를 $O(N)$ 의 시간 복잡도에 구할 수 있다. 먼저 조건 1., 2.를 필터링한 다음 위 알고리즘을 이용하면, 지그재그 수열이어야 한다는 조건 (조건 3.)까지 만족하는 가장 긴 부분수열을 구할 수 있다. 함수 f 의 $O(N^3)$ 가지의 값을 모두 구한 뒤 문제의 답을 구할 수 있으므로, 총 시간 복잡도는 $O(N^4)$ 이다.

부분문제 3

부분문제 2의 풀이를 개선하여 부분문제 3을 풀 수 있다. 부분문제 2의 풀이에서는 매번 비슷하게 계산되는 $O(N^3)$ 가지의 값을 구하고 있다는 것을 관찰할 수 있다. 슬라이딩 윈도우 기법을 사용하여 중복되는 계산들을 최적화할 수 있으며, $f(x, y, y+1), f(x, y, y+2), \dots, f(x, y, N)$ 까지의 값들을 한 번의 반복으로 구할 수 있다. 시간 복잡도가 $O(N^3)$ 이 되어 부분문제 3을 해결할 수 있다.

부분문제 4

부분문제 4부터의 부분문제들을 해결하려면 부분문제 3까지와 다른 관점에서 문제를 바라보아야 한다. 각 f 의 값들을 일일이 계산한 뒤 합쳐서 g 의 값을 만들지 말고, g 의 값을 직접 계산해보자. 여기에는 더 많은 관찰이 필요하다.

부분문제 2에서 제시한 가장 긴 지그재그 부분수열을 계산하는 알고리즘을 관찰하면, 수열 $S_1, \dots, S_K$ 의 가장 긴 지그재그 부분수열의 길이를 계산하는 간단한 공식을 알 수 있다. (a)가 실행될 때마다 arr 의 길이가 하나씩 증가한다. 다른 말로, 총 반복 횟수 K 에서 (b)가 실행되는 횟수를 빼면 알고리즘이 끝난 뒤 arr 가 된다. 즉, $A_y, \dots, A_z$ 에서 x 이하인 수들을 모아서 만든 수열을 C 라 하면, $f(x, y, z)$ 의 값은

1. C 의 길이
2. $C_i < C_{i+1} < C_{i+2}$ 를 만족하는 i 의 개수
3. $C_i > C_{i+1} > C_{i+2}$ 를 만족하는 i 의 개수

일 때 1. - 2. - 3.가 된다. $f(x, y, z)$ 의 합으로 정의되는 $g(x)$ 도 1.를 모두 합한 값에서 2., 3.를 모두 합한 값을 빼서 계산할 수 있다. 이 때, 고정된 x 에 대해 가능한 y, z 의 값을 모두 고려해 계산한다는 점에 유의하라. 즉, 위에서 정의한 부분수열 C 는, 모두 A 에서 x 이하인 수들을 모은 부분수열을 잘라 만든 수열이다. A 에서 x 이하인 수를 모은 배열을 $B_1, \dots, B_K$ 로 두고, 각 원소 B_j 의 배열 A 에서의 위치를 P_j 로 두자.

$g(x)$ 와 같은 값을 계산할 때에 유용하게 쓰이는 계산법은, 수열의 각 원소에 대해 해당 원소가 세어지는 횟수를 합하는 것이다. 각 j 에 대해 B_j 를 몇 번 세게 되는지 계산하자. 먼저, B_j 를 1.에서 세는 경우는 $y \leq P_j \leq z$ 인 경우이며, 총 $P_j \times (N - P_j + 1)$ 번이다. 2.에서 세는지 여부는 $B_j < B_{j+1} < B_{j+2}$ 인지에 달려 있다. 이 조건이 성립한다면, B_j 를 $P_j \times (N - P_{j+2} + 1)$ 번 세고, 그렇지 않다면 2.에서는 B_j 를 세지 않는다. 3.에서도 $B_j > B_{j+1} > B_{j+2}$ 가 성립한다면 $P_j \times (N - P_{j+2} + 1)$ 번 세고, 그렇지 않으면 세지 않는다.

위를 각각 계산한 다음 1. - 2. - 3.를 하면 $g(x)$ 에서 B_j 의 ‘기여도’가 된다. $B_1, B_2, \dots, B_K$ 에 대해 이 값들을 모두 합하면 $g(x)$ 의 값을 얻을 수 있다. 이를 별도의 최적화를 거치지 않고 구현하면 $O(N^3)$ 의 시간 복잡도로 부분문제 3까지 해결할 수 있고, 집합 자료구조 (예를 들어 C++의 `std::set`)을 이용해 구현하면 $O(N^2 \log N)$ 의 시간 복잡도로 부분문제 4까지 해결할 수 있다.

부분문제 5

부분문제 4와 같이 각 원소의 ‘기여도’를 세는 접근법은 이 풀이를 선형으로 개선할 때에도 그대로 써먹을 수 있다.

x 의 값이 $1, 2, \dots, N$ 일 때 $g(x)$ 의 값을 순서대로 계산하는 과정에서, x 의 값이 1 증가해서 $x+1$ 이 되는 순간을 일종의 쿼리로 생각할 수 있다. x 가 증가하여 $x+1$ 이 되는 것은 수열 B 의 어딘가에 $x+1$ 이 하나 추가된 것과 같다. 따라서 $g(x+1)$ 의 값을 계산할 때에는, $g(x)$ 의 값에서 새로 추가된 $x+1$ 의 ‘기여도’를 합해주면 된다. 이는 부분문제 4에서 소개한 방법으로 계산할 수 있다. 수열의 가운데에 $x+1$ 이 추가됨으로써 (추가된 위치 주변의) 다른 원소의 ‘기여도’가 변화하는데, 이 또한 계산에 포함시켜야 한다는 점 잊지 말자.

위 아이디어를 이용해 부분문제 4를 푸는 시간 복잡도에서 N 을 하나 제거할 수 있으며, $O(N \log N)$ 의 시간 복잡도로 전체 문제를 해결할 수 있다.

“잔디밭의 개미굴” 문제 풀이

작성자: 구재현

편의상 그래프 이론의 용어를 사용한다. 즉, 개미굴은 그래프이고, 방은 정점, 통로는 간선이다.

부분문제 1

모든 배정의 경우의 수는 2^N 가지이다. 고로, 그래프가 주어졌을 때, 최대의 개미가 들어가는 배정을 $O(2^N \times N)$ 시간에 찾을 수 있다. 원래 그래프에 대한 최적 배정을 찾고, 모든 $N(N-1)/2$ 개의 추가 간선에 대해서 여전히 최적 배정의 크기가 유지되는가를 확인한다. 시간 복잡도는 $O(2^N \times N^3)$ 이다.

부분문제 3

최적 배정의 크기를 찾는 문제는 흔히 **최대 독립 집합** 문제라고 하여 일반적인 그래프에서 효율적으로 푸는 것이 불가능하다. 개미굴이 트리라는 점을 사용하여 문제를 효율적으로 해결해야 한다.

다음과 같은 사실이 크게 도움이 된다:

Theorem. $|V| \geq 2$ 인 임의의 연결 그래프 $G = (V, E)$ 와 어떠한 차수 1 정점 v 에 대해, v 를 포함하는 최대 독립 집합이 존재한다.

Proof. 그렇지 않다고 가정하자. 임의의 최대 독립 집합을 골랐을 때, 이 집합에는 v 에 인접한 유일한 정점 w 가 포함된다. 만약 포함되지 않으면 v 를 넣을 수 있고 최대성에 모순이기 때문이다. 집합에서 w 를 지우고 v 를 넣는다고 해도, 여전히 인접한 두 정점이 모두 선택되는 일은 없다. 고로 v 를 포함하는 최대 독립 집합을 찾았고, 가정에 모순이다. ■

이를 사용하면 간선이 추가되지 않은 트리에서는 쉽게 문제를 해결할 수 있다. DFS를 사용하여 재귀적으로 해결하는데, 리프에서는 무조건 해당 정점을 독립 집합에 추가하고, 그 외 노드들에 대해서는 만약 자식이 하나라도 선택되었다면 추가하지 않고, 그렇지 않다면 추가할 수 있기 때문이다. 이 알고리즘은, 임의의 차수 1 이하인 노드를 찾아서 이 노드와 인접한 노드를 반복적으로 지워주는 알고리즘과 동치이기 때문에, 정당하다.

트리에 간선이 하나 추가되었을 때, 그래프는 하나의 사이클의 각 노드에 트리가 달린 꼴을 한다. 각 사이클 노드에 달린 트리에 대해 DFS로 독립적으로 해결한다. 이 경우, 사이클에 속하지 않은 노드들에 대해서는 차수 1인 노드가 반복적으로 지워진 것과 동치이기 때문에 올바른 배정이지만, 사이클에 속한 노드들은 인접한 간선들을 억지로 무시한 것이기 때문에 올바른 배정이 아니고, 실제로 사이클 상 인접한 노드가 골라지는 배정을 얻게 될 것이다.

이렇게 얻은 배정에서 어떠한 사이클 상 노드가 골라지지 않았다면, 이 노드에 인접한 사이클 밖 노드가 선택되었기 때문에, 이 노드는 제거된 것으로 취급하여도 좋다. 사이클 상 노드가 골라졌다면, 이 노드는 제거되지 않았다고 생각할 수 있다. 여기서 두 가지 케이스가 존재한다.

- 만약 모든 사이클 상 노드가 제거되지 않았다면, 사이클 밖 배정을 고민할 필요가 없으니 사이클 상에서 독립 집합을 고르면 된다. 즉, 사이클 크기를 2로 나눈 몫이 답에 추가된다.
- 사이클 상 제거된 노드가 있다면, 사이클 안에서 제거되지 않은 노드들은 선의 형태를 이룬다. 각각의 선에서 독립 집합을 고른 후 합쳐주면 된다. 즉, 각 선의 크기를 2로 나눈 값을 반올림하고, 답에 더한다.

트리, 그리고 트리에 간선이 하나 추가된 경우 모두 $O(N)$ 에 최적 배정의 크기를 구할 수 있다. 이를 $O(N^2)$ 번 반복하니 시간 복잡도는 $O(N^3)$ 이다.

부분문제 4

부분문제 4는 $O(N^2)$ 알고리즘을 구현하였을 때 정답을 받을 수 있게 구성되었다. (효율적인 알고리즘은 부분문제 5 이상도 맞을 수 있다.) 이 문제의 $O(N^2)$ 풀이는 다양하지만, 여기서는 의도된 만점 풀이의 방향에 있는 $O(N^2)$ 풀이를 설명한다.

다음 사실을 사용한다. 주어진 트리를 T 라 하고, T 에 간선 (i, j) 가 추가된 것을 $T + (i, j)$ 라고 하자.

Theorem. $S = \{v | v \in V(T), v \text{ 가 모든 최대 독립 집합에 속함}\}$ 이라고 하자. (i, j) 가 평화로운 쌍이 아니라는 것과, $i \in S, j \in S$ 임이 동치이다.

Proof. $\rightarrow$: (i, j) 가 평화로운 쌍이 아니라면, T 의 모든 최대 크기 독립 집합을 보았을 때, i 와 j 는 모두 그 집합에 속해야 한다. 그렇지 않다면 해당 집합을 $T + (i, j)$ 의 독립 집합으로 취할 수 있어, 평화로운 쌍이 아니라는 가정에 모순이기 때문이다.

$\leftarrow$: 대우명제를 증명한다. (i, j) 가 평화로운 쌍이라면, $T + (i, j)$ 의 최대 독립 집합의 크기가 T 의 최대 독립 집합의 크기와 동일하다. $T + (i, j)$ 의 독립 집합은 T 의 독립 집합이니, 모든 T 의 최대 독립 집합을 살펴보았을 때, 이 중 $T + (i, j)$ 에서도 최대 독립 집합인 것이 존재한다. 이는, i, j 중 하나는 이 집합에 속하지 않는다는 것이다. 고로 i, j 가 모두 S 에 속할 수 없다. ■

Theorem을 사용하면, S 의 원소를 구했을 때 답을 얻는 것은 간단하다. S 의 원소를 구하기 위해서는, 각 정점을 지워보고, 지웠을 때 생기는 서브트리들에 대해 부분문제 3의 방법으로 최대 독립 집합의 크기를 구하여, 그 크기의 합이 원래 독립 집합의 크기보다 작은지 확인하면 된다. 시간 복잡도는 $O(N^2)$ 이다.

부분문제 7

부분문제 3에서 트리의 최대 독립 집합을 구하는 과정을 살펴보면, 알고리즘은 임의의 정점을 루트로 잡은 후, 계산 과정에서 트리의 모든 서브트리에 대해서 최대 독립 집합의 크기를 구한다는 사실을 알 수 있다. 고로, 부분문제 4에서 우리가 구하는 지웠을 때 생기는 서브트리 중, 루트 방향으로 향하는 서브트리가 아닌 모든 서브트리에 대해서는 $O(N)$ 시간에 답을 전부 구해놓을 수 있다. 이 정보를 구해 두면, 모든 루트 방향으로 향하는 서브트리에 대해서도 문제를 해결할 수 있다.

구체적으로, 부분문제 3의 알고리즘은 각 노드 v 에 대해서 v 의 서브트리의 독립 집합 크기, 그리고 v 가 그 독립 집합에 선택되었는지 여부를 계산한다. 여기서 우리가 빠르게 계산하고 싶은 값은, 각 노드 v 에 대해서, v 를 루트로 하였을 때 $par(v)$ 의 서브트리에서 독립 집합의 크기, 그리고 $par(v)$ 가 그 독립 집합에 선택되었는지의 여부이다.

이 값을 루트에서부터 top-down으로 계산하자. 각 노드 v 에 대해서, 위 답의 계산 결과는 거의 $par(v)$ 의 서브트리 계산 결과와 일치하는데, 다른 점은 1) v 로 가는 서브트리를 빼야 한다 2) $par(v)$ 에서 $par(par(v))$ 로 가는 서브트리를 더해야 한다는 점이다. 2) 의 서브트리 값은 계산 순서에 따라서 이미 알고 있고, 1) 의 서브트리 값은 초기에 다 구해 놓았으니, 이제 이 변환값을 계산해야 한다. 이 값을 $deg(v)$ 에 계산하면, 전체 시간 복잡도가 $O(N^2)$ 가 된다. $onchild(v)$ 를, v 의 자식 중 독립 집합에 포함된 자식의 수라고 하자. 이 값을 맨 처음 서브트리에 대해 문제를 해결할 때 전처리해 두면, $par(v)$ 가 v 가 루트일 때 독립 집합에 들어가는 지 여부를 빠르게 계산할 수 있다. 이 값이 저장되면, 독립 집합 크기는 이 값과 상술한 계산 결과들을 조합하여 계산할 수 있다. 고로 각 노드에 대해 필요한 값을 $O(1)$ 에 계산할 수 있다. 시간 복잡도는 $O(N)$ 이다.

“바보 자물쇠” 문제 풀이

작성자: 구재현

서술의 편의를 위해 각 알파벳 소문자를 숫자에 대응시킨다. 예를 들어, 'a' 는 0, 'b' 는 1.. 'z' 는 25 와 같은 식이다. 등장할 수 있는 서로 다른 알파벳의 수를 Σ 라고 표기한다. 예를 들어, 만점 서브테스크에서 $\Sigma = 26$ 이다.

부분문제 1

자물쇠가 풀리는 경우는 총 $N + 1$ 가지가 있는데, 맨 앞 i 개 문자가 0 이고 뒤 $N - i$ 개 문자가 1 인 경우 이다. 각 경우에 대해서, 두 문자열이 다른 위치의 개수를 센 후, 그 최소값을 출력하면 된다. 시간 복잡도는 $O(N^2)$ 이다.

부분문제 2

부분문제 1의 풀이를 최적화한다. 모든 $0 \leq i \leq N$ 에 대해, 맨 앞 i 개 문자 중 1 의 개수, 맨 뒤 $N - i$ 개 문자 중 0 의 개수를 빠르게 셀 수 있으면 된다. 이는 부분 합을 계산하여 해결할 수 있다. 매 쿼리마다 부분합을 계산한 후 최소값을 계산하면, $O(QN)$ 시간에 문제가 해결된다.

부분문제 3

자물쇠가 풀리는 시점에서 1 이 등장하는 위치가 구간 $i, i+1, \dots, j-1$ 을 이룬다고 하자. j 이후 위치에서는 2 가 등장할 것이고, $i-1$ 이전 위치에서는 0 이 등장할 것이다. $F(i)$ 를 $i-1$ 이전 위치에 0 이 적히도록 하는 최소 시간, $G(j)$ 를 j 이후 위치에 2 가 적히도록 하는 최소 시간이라고 정의하자. 답은 $F(i), G(j)$, 그리고 몇가지 부분합으로 표현할 수 있으며, F, G 역시 모두 부분합으로 계산할 수 있다. 식이 다소 복잡하여 자세한 설명은 생략한다. $O(QN)$ 시간에 문제가 해결된다.

부분문제 5

어떠한 문자 x 를 새로운 문자 y 로 바꾸는 데 드는 연산 수는 $|x - y|$ 임을 관찰하자.

동적 계획법을 사용한다. $dp[i][j]$ 를, i 번 문자를 j 이하의 어떤 값으로 설정했을 때 드는 최소 연산 수라고 정의하자. 상태 전이는 두 가지이다:

- i 번 문자를 j 로 설정한다: 이 경우의 연산 수는 $dp[i-1][j] + |j - S[i]|$ 이다.
- i 번 문자를 $j-1$ 이하의 값으로 설정한다: 이 경우의 연산 수는 $dp[i][j-1]$ 이다.

이 동적 계획법은 간단하게 $O(N\Sigma)$ 시간에 계산할 수 있다. 시간 복잡도는 $O(QN\Sigma)$ 이다.

다른 동적 계획법을 사용하면 $O(QN\Sigma^2)$ 나 $O(QN\Sigma \log N)$ 의 시간 복잡도가 나올 수 있다. 이 경우에는 부분문제 4가 통과되지만, 부분문제 5는 통과되지 않을 수 있다.

부분문제 6

KOI 2014 중등부에 출제된 금광 문제와 유사한 방법으로 해결할 수 있다.

세그먼트 트리 자료구조를 사용하자. 트리의 각 노드에는 다음과 같은 정보를 저장한다:

- 구간에 있는 0 의 개수
- 구간에 있는 1 의 개수
- 구간의 앞을 0 으로 채우고, 뒤를 1 으로 채웠을 때 필요한 최소 연산 수

두 노드가 주어졌을 때, 구간에 있는 0 의 개수와 1 의 개수는 쉽게 합칠 수 있다. 최소 연산 수를 합칠 때는, 0 과 1 의 분기점 을 어디로 설정하는 지에 따라 두 가지 경우가 있다. 만약 0 과 1 의 분기점 이 왼쪽 구간에서 형성된다면, 오른쪽 구간은 1 로 결정되니 추가되는 연산 수는 오른쪽 구간의 0 의 개수이다. 분기점 이 오른쪽 구간에서 형성된다면, 같은 이유로 추가되는 연산 수는 왼쪽 구간의 1 의 개수이다. 이 둘 중 최솟값을 취하면 된다.

이러한 세그먼트 트리를 구성하면 갱신 쿼리를 효율적으로 처리할 수 있다. 시간 복잡도는 $O(N + Q \log N)$ 이다.

부분문제 7

부분문제 6의 풀이를 확장하면 부분문제 7을 해결할 수 있다. 세부적인 사항이 다소 복잡하여 설명은 생략한다. 부분문제 8의 풀이가 비효율적으로 구현되었을 경우에도 부분문제 7에서 점수를 얻을 수 있다.

부분문제 8

부분문제 6의 풀이를 일반화할 경우, 행렬과 유사한 형태의 구조를 얻을 수 있다. 세그먼트 트리 자료구조의 노드에, $\Sigma \times \Sigma$ 의 크기를 가지는 2차원 배열 $Cost[][]$ 를 정의하자. 이 때, $Cost[i][j]$ 는 구간의 맨 앞 원소를 i 로 설정하고, 맨 뒤 원소를 j 이하로 설정하며 해당 구간을 증가하게 만들려 할 때 필요한 최소 연산 수이다.

어떠한 부모 노드의 $Cost[i][j]$ 를 계산할 때는, 오른쪽 자식 노드의 맨 앞 원소 k 를 고정하면, 왼쪽 자식 노드의 $Cost[i][k]$ 와 오른쪽 자식 노드의 $Cost[k][j]$ 의 합 만큼의 연산 수가 필요함을 알 수 있다. 이를 모든 $i \leq k \leq j$ 에 대해 순회하며 최솟값을 구하면 된다. 시간 복잡도는 $O((N + Q \log N)\Sigma^3)$ 이다.

부분문제 9

만점을 받기 위해서는 전체 문제를 $\Sigma - 1$ 개의 부분문제 6 인스턴스로 변환시키는 아이디어가 필요하다. 모든 $1 \leq i \leq \Sigma - 1$ 에 대해, $bin(S, i)$ 를 $S_j \geq i$ 일 때 1 이고 아닐 때 0 인 길이 N 의 이진 문자열이라고 정의하자. 또한, $cost(S)$ 를 어떠한 수열 S 를 단조증가하게 만드는 최소 연산 수라고 하자. 다음 사실이 성립한다.

Theorem. $cost(S) = \sum_i cost(bin(S, i))$

Proof. $cost(S) \geq \sum_i cost(bin(S, i))$: $cost(S)$ 의 답을 주는 어떠한 최적해 $opt(S)$ 를 생각해 보자. 이제 각각의 i 에 대해 $bin(S, i)$ 를 $bin(opt(S), i)$ 로 변환할 경우, 이들 중 j 번 위치에서 다른 값을 가지는 i 의 개수는 정확히 $|S_j - opt(S)_j|$ 개임을 관찰할 수 있다. 고로 우변은 $cost(S)$ 초과일 수 없다.

$cost(S) \leq \sum_i cost(bin(S, i))$: 각 i 에 대한 최적해 $opt(bin(S, i))$ 를 생각해 보자. 또한, $zero_i$ 를 $opt(bin(S, i))$ 에 있는 0 의 개수라고 하자.

만약 $zero_i$ 가 단조증가 수열이라면, i 번 문자를 S 의 $[zero_i, zero_{i+1})$ 구간에 배정함으로써 $cost(bin(S, i))$ 의 최적해를 구성할 수 있다 ($zero_0 = 0, zero_\Sigma = N$). 이 수열은 증가하고, $|S_j - opt(S)_j|$ 는 이 수열 중 j 번 문자가 달라진 수열의 개수와 일치한다. 고로, 이 경우 $cost(bin(S, i))$ 의 합의 연산 수로 S 를 단조증가하게 변환했다고 할 수 있다.

$zero_i$ 가 단조증가 수열이 아니라고 하자. 즉, $zero_i > zero_{i+1}$ 인 위치 i 가 존재한다고 하자. 이 경우 $opt(bin(S, i))$ 와 $opt(bin(S, i+1))$ 의 최적해를 서로 바꿔도 답이 늘어나지 않음을 증명할 수 있다. $bin(S, i+1)$ 이 $bin(S, i)$ 보다 구간 $[zero_{i+1}, zero_i)$ 에서 더 많은 0 을 포함하고 있기 때문이다. 고로 이 경우 $zero_i$ 와 $zero_{i+1}$ 을 바꿔줄 수 있고, 버블 정렬의 요령으로 이를 반복해주면 연산 수에서 손해를 보지 않고 $zero_i$ 가 단조증가 수열이라고 가정할 수 있다. ■

Theorem에 의해, 부분문제 6의 풀이를 그대로 적용해서 문제를 해결할 수 있다. 시간 복잡도는 $O((N + Q \log N)\Sigma)$ 이다.

여담

사이클이 없는 방향성 그래프 $G = (V, E)$ 의 각 정점에 $N = |V|$ 개의 수 $A_1, A_2, \dots, A_N$ 이 적혀 있을 때, G 의 모든 간선 $u \rightarrow v$ 에 대해 $A_u \leq A_v$ 를 만족하게끔 하는 문제를 **Isotonic Regression** 이라고 한다. 이 문제는, 그래프를 $i \rightarrow i+1$ 형태의 단순한 선 그래프로 생각하고, 숫자를 바꾸는 비용을 $|x - y|$ (L_1 metric) 으로 두었을 때의 Isotonic Regression을 푸는 문제라고 생각할 수 있다.

선 그래프의 L_1 Isotonic Regression은 잘 알려진 문제이고 (BOI 2004 Sequence), 경시대회에서 *Slope trick* 이라고 부르는 방법으로 해결된다. Leftist heap을 사용한 풀이도 있지만, 다소 복잡한 풀이이다. 이 문제에서 제안된 접근을 사용하면, *Slope trick* 과 완전히 다른 방향에서 $O(N \log N)$ 풀이를 얻을 수 있다.

이러한 접근은 기존 *Slope trick* 보다 더 직관적이고 (특히 최적해의 복원), 제한된 형태의 업데이트를 지원할 수 있으며, 더 다양한 문제를 해결할 수 있다는 장점이 있다. 만약에 Isotonic Regression을 $A_i \in \{0, 1\}$ 이라는 제약 조건 하에서 해결한다면, 트리에서는 DP를 사용하여 $O(N)$ 시간에, DAG에서는 Network flow를 사용하여 $O(N^{1+o(1)})$ 시간에 해결할 수 있다. 이러한 풀이는 N 번 반복하여 원래 문제의 풀이로 복원되거나, A_i 가 작을 때 갱신을 지원하는 dynamic algorithm을 사용하여 (예를 들어, 트리의 경우에는 Top tree를 사용하여 여러 갱신이 지원된다) 다양한 방향의 효율적인 알고리즘을 얻을 때 사용할 수 있다.

이 문제는 IOI 2023에 Call for tasks를 통하여 제안되었던 문제이다.